

Rethinking Patch Based Multivariate Time Series Forecasting with Semantic Structured Partitioning

Jiazhe Wang^{a,1}, Zhiquan Huang^{b,1}, Linjing Xue^a, Ming Liu^a, Meiwen Li^a and Ruijuan Zheng^{b,*}

^aSchool of Software, Henan University of Science and Technology, Luoyang, 471000, Henan, China

^bSchool of Information Engineering, Henan University of Science and Technology, Luoyang, 471023, Henan, China

ARTICLE INFO

Keywords:

Multivariate time series forecasting
Semantic structured partitioning
Transfer entropy graph
Importance aware routing

ABSTRACT

Multivariate time series forecasting (MTSF) is a fundamental task in many real world applications. Existing patch based forecasting methods generally fall into three categories: fixed partitioning, multi-scale partitioning, and extendable partitioning. Fixed partitioning often breaks meaningful temporal boundaries, multi-scale partitioning may introduce redundant representations across scales, and extendable partitioning improves flexibility but still lacks an explicit mechanism for organizing semantic structure and modeling interactions among heterogeneous temporal patterns. To address these limitations, we propose SCPaT, a Transformer based framework built on semantic structured partitioning. SCPaT first decomposes input sequences into semantically consistent units through adaptive semantic unit generation, then constructs a dynamic semantic graph to model directed dependencies among these units and organize them into higher order semantic blocks. Based on these structured representations, an importance aware routing mechanism adaptively dispatches different semantic blocks to different experts for customized modeling. Extensive experiments on 12 real world datasets demonstrate the effectiveness of SCPaT.

1. Introduction

Multivariate time series forecasting (MTSF) plays an important role in various domains (Qiu et al., 2024), including electricity load forecasting (Gasparin et al., 2022), weather forecasting (Karevan and Suykens, 2020), and traffic flow forecasting (Lippi et al., 2013). Recent advances in deep learning have led to a wide range of forecasting architectures for MTSF (Huang et al., 2025). Among them, patch based modeling has emerged as an effective paradigm for capturing complex temporal dependencies (Cao et al., 2025). By partitioning long sequences into temporal patches, these methods reduce modeling complexity while preserving fine grained local patterns (Zhang and Yan, 2023a). However, the effectiveness of these models hinges critically on how the input sequence is partitioned, a design choice that, as we will show, has profound implications for semantic preservation. Consequently, the design of patch partitioning strategies has become a key consideration in patch based Multivariate time series forecasting.

Existing patch partitioning strategies can be broadly categorized into three types: fixed partitioning (Nie et al., 2023), multi-scale partitioning (Wu et al., 2022), and extendable partitioning (Huang et al., 2024). Among these strategies, fixed partitioning is the most widely adopted in patch based forecasting. It divides a time series into equal length segments and treats each segment as an input unit,

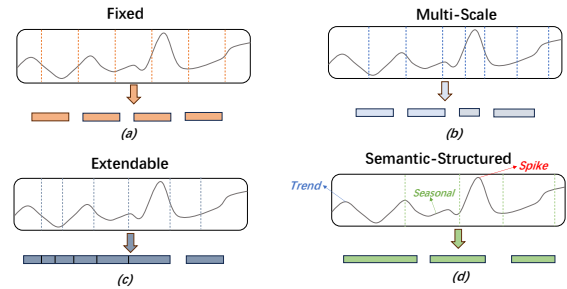


Figure 1: Comparison of four patch based schemes: fixed partitioning, multi-scale partitioning, extendable partitioning, and the proposed semantic structured partitioning.

thereby reducing the effective sequence length and simplifying the modeling process. As illustrated in Figure 1(a), this design is computationally efficient and performs well on time series with regular and predictable patterns. However, because it relies on uniform segmentation, fixed partitioning often fails to align with meaningful temporal boundaries in nonstationary sequences, particularly in the presence of trend shifts or abrupt events (Kazemi et al., 2020). This limitation restricts its ability to represent complex temporal dynamics and motivates the development of more flexible partitioning strategies.

To overcome the limited flexibility of fixed partitioning, researchers have proposed multi-scale partitioning. This strategy partitions the time series at multiple levels of granularity, so that scale specific representations can be modeled separately and then integrated. As shown in Figure 1(b), this strategy provides a richer characterization of temporal dynamics by capturing both local short term fluctuations

*Corresponding author

✉ 231451080320@stu.haust.edu.cn (J. Wang);

zqhuang@stu.haust.edu.cn (Z. Huang);

ljxue@stu.haust.edu.cn (L. Xue); liuming@haust.edu.cn

(M. Liu); mwli@stu.haust.edu.cn (M. Li);

zhengruijuan@haust.edu.cn (R. Zheng)

ORCID(s):

¹These authors contributed equally to this work.

and global long term trends (Ye and Ma, 2023). However, when similar temporal patterns are captured at multiple granularities, the corresponding representations may overlap substantially across scales, thereby introducing redundancy during cross scale integration (Hu et al., 2025; Liu et al., 2025b). Such redundancy may in turn encourage the model to learn superfluous features. This problem becomes more pronounced in datasets with complex and overlapping dependencies, where heterogeneous dynamic patterns are more difficult to disentangle. These limitations motivate the development of more adaptive partitioning mechanisms.

Extendable partitioning further increases flexibility by allowing the length of each temporal segment to vary according to data characteristics (Zhang et al., 2025; Liu et al., 2025a). By adapting segment granularity to local dynamics, this strategy better preserves temporal boundary information and captures local semantic patterns. As illustrated in Figure 1(c), such adaptive partitioning provides greater flexibility for modeling the diversity and complexity of temporal variations. Nevertheless, existing extendable partitioning methods typically determine segmentation granularity using heuristic rules, which limits their ability to handle complex higher order dependencies. In scenarios characterized by strong nonlinear dependencies or multiple coexisting temporal patterns, these methods often struggle to model interactions among heterogeneous patterns, which results in suboptimal forecasting performance. These observations suggest that improving partition flexibility alone is insufficient for modeling the semantic structure underlying complex temporal dynamics.

Despite the advances achieved by existing methods, a critical research gap remains fundamentally unaddressed: none of the current partitioning strategies explicitly accounts for the intrinsic semantic coherence of temporal segments, nor do they model the higher order structural interactions among heterogeneous temporal patterns. In essence, all these approaches treat patch generation as a geometric preprocessing step rather than as an integral semantic modeling task. This deficiency is particularly detrimental when trends, periodicities, and abrupt fluctuations coexist in the same time series, as the model lacks a principled and feasible mechanism to disentangle these diverse components and establish structured relationships among them. Therefore, we argue that the core challenge of patch-based forecasting lies not in pursuing greater partition flexibility, but in constructing a representation system that embeds semantic structure.

Based on the above analysis, we propose SCPaT, a Transformer based framework for MTSF that models dynamic semantic structure through semantic structured partitioning. Specifically, SCPaT extracts fundamental temporal components from raw time series and organizes them into hierarchical semantic blocks, which provide multilevel semantic abstractions for forecasting. These semantic blocks are then incorporated into the attention mechanism through an importance aware routing module, which enables the model to emphasize informative representations and capture

dependencies across different semantic types. Extensive experiments on multiple benchmarks show that SCPaT achieves state of the art results in both long term and short term forecasting tasks. The main contributions of this work are summarized as follows:

- **Systematic Analysis.** We present a systematic analysis of three representative patch partitioning strategies for time series forecasting and show that their limitations reflect the lack of an explicit semantic structured partitioning mechanism.
- **Innovative Framework.** We propose SCPaT, a Transformer based framework that integrates hierarchical semantic blocks and an importance aware routing mechanism into attention, enabling more effective modeling of key dependencies in MTSF.
- **Extensive Evaluations.** Extensive experiments on long term and short term forecasting benchmarks demonstrate that SCPaT achieves state of the art performance across a wide range of datasets.

2. Related Work

2.1. Transformer based Time Series Forecasting

Transformer-based models have become widely used in MTSF due to their strong parallel modeling capability and ability to capture long-range dependencies (Qiu et al., 2026; Liu et al., 2022). Representative methods have improved forecasting performance from different perspectives. For instance, iTransformer (Liu et al., 2024) enhances cross-variable dependency modeling by reorganizing tokens along the variable dimension. Pathformer (Chen et al., 2024) captures both local and global temporal contexts using multiscale patch partitioning with an adaptive path strategy. Fredformer (Piao et al., 2024) mitigates high-frequency bias through frequency debiasing. However, these methods are still limited in their ability to explicitly characterize heterogeneous dynamics within patches and capture interactions across different temporal patterns (Yang et al., 2025). This limitation becomes particularly evident when trends, periodicity, and abrupt fluctuations coexist in the same time series. Such scenarios require mechanisms that can distinguish local semantic differences and model complex dependencies more effectively.

2.2. Patch Based Time Series Forecasting

Patch based methods have become an important direction in time series forecasting, with early studies primarily relying on fixed partitioning strategies (Wu et al., 2025; Chen et al., 2024). PatchTST (Nie et al., 2023), for example, segments time series into equal length patches to improve representation efficiency. However, fixed partitioning often fails to preserve meaningful temporal boundaries, which may lead to the loss of salient local patterns and semantic inconsistency within patches. To address this issue, subsequent studies have explored multi-scale

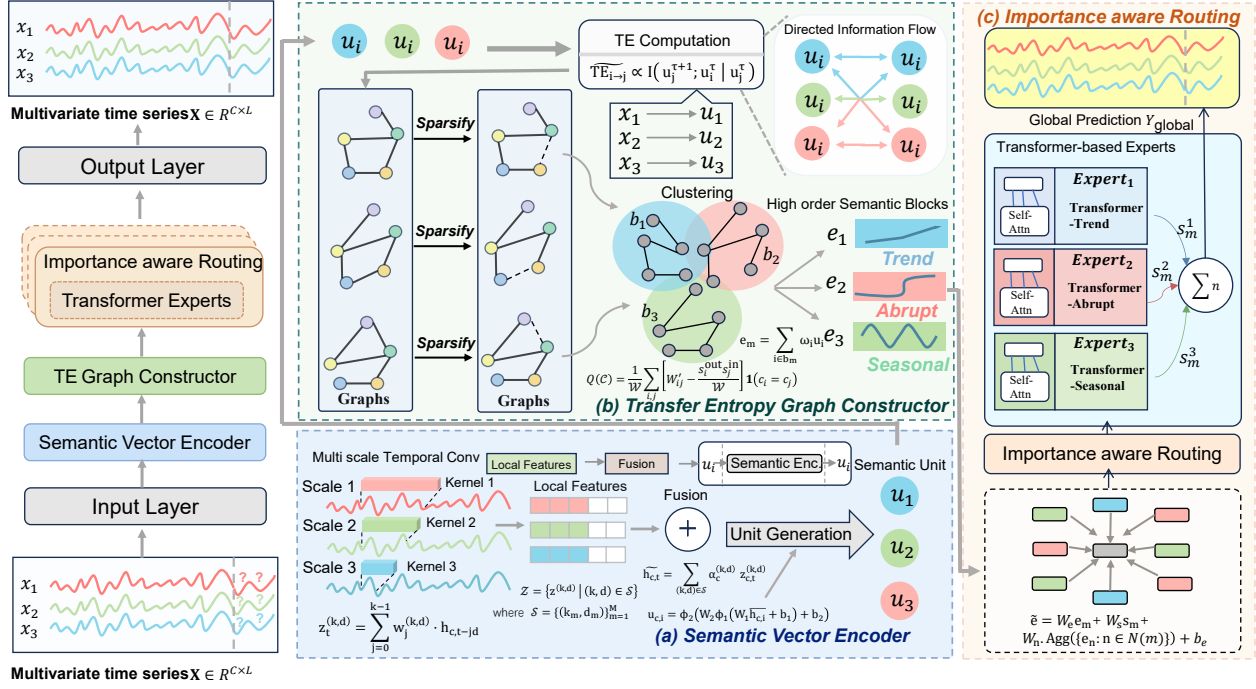


Figure 2: SCPaT architecture consists of: (a) Semantic Vector Encoder, which encodes time series into semantic units; (b) Transfer Entropy Graph Constructor, quantifies directed dependencies and constructs a dynamic semantic graph; and (c) Importance aware Routing, allocates customized processing strategies to different semantic blocks for effective modeling.

partitioning strategies. For instance, TimesNet (Wu et al., 2022) models temporal patterns across different scales, while FEDformer (Zhou et al., 2022) integrates local and global trends under a frequency domain decomposition framework. Extendable partitioning has further emerged as a flexible alternative, with HDMixer (Huang et al., 2024) introducing length adaptive patches that dynamically adjust segment boundaries, improving semantic integrity and modeling flexibility. Despite these advances, existing patch based methods remain limited in their ability to explicitly capture fine grained semantic variation and higher order temporal dependencies. All of them formulate patch generation as a geometric partitioning problem, whereas our SCPaT redefines it from a semantic structure perspective.

3. Proposed Methodology

3.1. Preliminaries

In multivariate time series forecasting, the input is denoted by $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_C\} \in \mathbb{R}^{C \times L}$, where C is the number of variables and L is the length of the historical lookback window. Here, $\mathbf{x}_c \in \mathbb{R}^L$ denotes the historical observations of the c -th variable. The goal is to learn a forecasting function $f_\theta(\cdot)$ that maps the historical sequence to future values $\hat{\mathbf{Y}} \in \mathbb{R}^{C \times H}$, where H is the prediction length. Formally, $\hat{\mathbf{Y}} = f_\theta(\mathbf{X})$. We partition the historical sequence of each variable into $N = \lfloor L/s \rfloor$ non overlapping temporal patches, where s denotes the patch size. These patches are then embedded into a hidden space of dimension

d_h , yielding patch level representations. Based on these representations, we further construct semantic units, semantic graphs, and block wise routed experts to model structured dependencies for forecasting.

3.2. Semantic Vector Encoder

The semantic vector encoder maps the input sequence $\mathbf{X} \in \mathbb{R}^{C \times L}$ into a set of local semantic representations. It extracts temporal patterns at multiple scales, adaptively fuses the resulting features, and organizes the fused sequence into semantic units.

3.2.1. Multi scale Temporal Convolution

To capture local temporal variations at different resolutions, we apply multi scale temporal convolutions to the projected feature sequence. For the c -th variable, let $\mathbf{h}_c = \{\mathbf{h}_{c,t}\}_{t=1}^L$ denote its projected feature sequence, where $\mathbf{h}_{c,t} \in \mathbb{R}^{d_h}$ is the hidden representation at time step t . For a temporal branch with kernel size k and dilation factor d , the output at time step t is defined as:

$$\mathbf{z}_{c,t}^{(k,d)} = \sum_{j=0}^{k-1} w_j^{(k,d)} \mathbf{h}_{c,t-jd}, \quad (1)$$

where $\mathbf{z}_{c,t}^{(k,d)} \in \mathbb{R}^{d_h}$ denotes the feature extracted at scale (k, d) , $w_j^{(k,d)}$ is the j -th kernel coefficient, and zero padding is applied when $t - jd < 1$.

The corresponding branch output in compact form is:

$$\mathbf{Z}_c^{(k,d)} = \mathbf{W}^{(k,d)} * \mathbf{H}_c, \quad (2)$$

where $*$ denotes temporal convolution, $\mathbf{H}_c = [\mathbf{h}_{c,1}, \dots, \mathbf{h}_{c,L}]$, and $\mathbf{W}^{(k,d)}$ is the convolution kernel associated with scale (k, d) . The effective receptive field of this branch is defined as $r^{(k,d)} = 1 + (k - 1)d$, which is determined by the kernel size k and dilation factor d .

To capture temporal patterns at multiple granularities, we introduce a set of temporal branches $\mathcal{S} = \{(k_m, d_m)\}_{m=1}^M$, where M is the number of branches. The resulting multi scale feature set is defined as:

$$\mathcal{Z}_c = \{\mathbf{Z}_c^{(k,d)} \mid (k, d) \in \mathcal{S}\}. \quad (3)$$

3.2.2. Adaptive Multi scale Fusion

To integrate temporal patterns captured at different scales, we perform adaptive multi scale fusion over the branch set $\mathcal{S}$. The fused hidden vector at time step t is defined as:

$$\tilde{\mathbf{h}}_{c,t} = \sum_{(k,d) \in \mathcal{S}} \alpha_c^{(k,d)} \mathbf{z}_{c,t}^{(k,d)}, \quad (4)$$

where $\tilde{\mathbf{h}}_{c,t} \in \mathbb{R}^{d_h}$ denotes the fused representation of variable c at time step t , and $\alpha_c^{(k,d)}$ denotes the fusion weight assigned to branch (k, d) .

The fusion weights are obtained by normalizing learnable branch scores:

$$\alpha_c^{(k,d)} = \frac{\exp(\gamma_c^{(k,d)})}{\sum_{(k',d') \in \mathcal{S}} \exp(\gamma_c^{(k',d')})}, \quad (5)$$

where $\gamma_c^{(k,d)}$ is the trainable score associated with scale pair (k, d) for variable c . This formulation allows the encoder to adaptively emphasize more informative temporal scales for each variable. For convenience, we denote the fused sequence of variable c by $\tilde{\mathbf{H}}_c = [\tilde{\mathbf{h}}_{c,1}, \tilde{\mathbf{h}}_{c,2}, \dots, \tilde{\mathbf{h}}_{c,L}]$.

3.2.3. Semantic Unit Generation

Based on the fused sequence $\tilde{\mathbf{H}}_c$, we partition each variable into non overlapping semantic units and encode each unit into a compact representation. Specifically, a unit starting at position i with length l_i spans $[i, i + l_i - 1]$, and the next unit starts at position $i + l_i$.

To adapt the partition to local temporal variation, the unit length is selected from a candidate set $\mathcal{L} = \{1, \dots, L_{\max}\}$, where $L_{\max}$ is the maximum unit length. For a candidate segment $\tilde{\mathbf{H}}_c[i : i + l' - 1]$, we compute a scalar activation at each time step as $a_{c,t} = \frac{1}{d_h} \sum_{r=1}^{d_h} \tilde{h}_{c,t,r}$, where $\tilde{h}_{c,t,r}$ denotes the r -th dimension of $\tilde{\mathbf{h}}_{c,t}$. The local variance of the candidate segment is then defined as:

$$\text{Var}(\tilde{\mathbf{H}}_c[i : i + l' - 1]) = \frac{1}{l'} \sum_{t=i}^{i+l'-1} (a_{c,t} - \bar{a}_{c,i,l'})^2, \quad (6)$$

where $\bar{a}_{c,i,l'} = \frac{1}{l'} \sum_{t=i}^{i+l'-1} a_{c,t}$ is the segment mean.

Let $\mathcal{L}_i^\delta = \{l' \in \mathcal{L} \mid \text{Var}(\tilde{\mathbf{H}}_c[i : i + l' - 1]) > \delta\}$ denote the admissible length set at position i . The unit length is then

determined as:

$$l_i = \begin{cases} \min \mathcal{L}_i^\delta, & \text{if } \mathcal{L}_i^\delta \neq \emptyset, \\ L_{\max}, & \text{otherwise.} \end{cases} \quad (7)$$

where δ is a threshold controlling the sensitivity to local variation. This rule assigns longer units to locally stable regions and shorter units to rapidly changing regions. The variance criterion determines adaptive structural boundaries, while the semantic content is encoded by the learned multi scale representation.

Because the segment length l_i varies across units, we first map each segment to a fixed dimensional summary vector via mean pooling, i.e., $\bar{\mathbf{h}}_{c,i} = \frac{1}{l_i} \sum_{t=i}^{i+l_i-1} \tilde{\mathbf{h}}_{c,t}$. The semantic unit representation is then computed as:

$$\mathbf{u}_{c,i} = \phi_2(\mathbf{W}_2 \phi_1(\mathbf{W}_1 \bar{\mathbf{h}}_{c,i} + \mathbf{b}_1) + \mathbf{b}_2), \quad (8)$$

where $\mathbf{W}_1$ and $\mathbf{W}_2$ are learnable projection matrices, and $\mathbf{b}_1$ and $\mathbf{b}_2$ are learnable bias vectors.

Since the number of semantic units N_c may vary across variables, establishing correspondences among units from different variables is essential for the subsequent Transfer Entropy modeling. For each timestamp t , we take the semantic unit that covers t from each variable as the aligned unit set at that time step. This yields a sequence of aligned unit tuples across the temporal axis, enabling consistent dependency modeling among variables in the TE graph construction. For variable c , this procedure yields a unit sequence $\mathcal{U}_c = \{\mathbf{u}_{c,1}, \mathbf{u}_{c,2}, \dots, \mathbf{u}_{c,N_c}\}$, where N_c is the number of extracted units. All units are then re indexed into a unified set $\mathcal{U} = \{\mathbf{u}_n\}_{n=1}^{N_u}$, where $N_u = \sum_{c=1}^C N_c$. Each unit $\mathbf{u}_n$ is associated with its variable identity and temporal span for subsequent dependency modeling.

3.3. Transfer Entropy Graph Constructor

The transfer entropy graph constructor quantifies directed information flow among semantic units to build a dynamic semantic graph, which subsequently supports the formation of higher order semantic blocks. This design captures nonlinear and asymmetric dependency structures beyond simple correlation based relations.

3.3.1. Transfer Entropy Computation.

Traditional correlation based measures are often insufficient for modeling directed nonlinear dependencies, as they are typically symmetric and do not capture conditional information flow. We therefore introduce a differentiable transfer entropy inspired surrogate for directed conditional dependence between semantic units.

Specifically, for two semantic units $\mathbf{u}_i$ and $\mathbf{u}_j$, we use the following conditional dependence target:

$$\widetilde{TE}_{i \rightarrow j} \propto I(\mathbf{u}_j^{t+1}; \mathbf{u}_i^t \mid \mathbf{u}_j^t), \quad (9)$$

where $I(\cdot; \cdot \mid \cdot)$ denotes conditional mutual information, and τ indexes valid aligned unit transitions. This target captures

the additional predictive information provided by $\mathbf{u}_i^\tau$ beyond the self history $\mathbf{u}_j^\tau$.

To obtain a practical differentiable surrogate, we use τ for the aligned unit level step within a sample and b for the sample index in the mini batch. For each ordered pair (i, j) , let Ω_{ij} denote the set of valid aligned transitions (b, τ) in the current mini batch for which $\mathbf{u}_i^{(b, \tau)}$, $\mathbf{u}_j^{(b, \tau)}$, and $\mathbf{u}_j^{(b, \tau+1)}$ all exist, and let $M_{ij} = |\Omega_{ij}|$. We then define the directed dependency surrogate as:

$$\widehat{TE}_{i \rightarrow j} = \frac{1}{M_{ij}} \sum_{(b, \tau) \in \Omega_{ij}} \psi(\mathbf{u}_j^{(b, \tau+1)}, \mathbf{u}_i^{(b, \tau)}, \mathbf{u}_j^{(b, \tau)}), \quad (10)$$

where $\psi(\cdot)$ is a learnable sample level scoring function.

In practice, $\psi(\cdot)$ is instantiated as a 3-layer MLP with hidden dimensions $[3d_h, d_h, 1]$, where the input is the concatenation of $\mathbf{u}_j^{(b, \tau+1)}$, $\mathbf{u}_i^{(b, \tau)}$, and $\mathbf{u}_j^{(b, \tau)}$, and the output is a scalar dependency score. This network contains approximately $3d_h^2 + d_h$ parameters, accounting for less than 1% of the total model and incurring negligible inference overhead. For each valid transition $(b, \tau) \in \Omega_{ij}$, we form a triplet representation $\mathbf{q}_{ij}^{(b, \tau)}$ by concatenating $\mathbf{u}_j^{(b, \tau+1)}$, $\mathbf{u}_i^{(b, \tau)}$, and $\mathbf{u}_j^{(b, \tau)}$, and compute its score using $\text{MLP}_\psi(\mathbf{q}_{ij}^{(b, \tau)})$, where MLP_ψ outputs a scalar and is shared across all ordered pairs (i, j) .

To characterize the stability of the estimated dependency, we further compute its sample standard deviation over the same valid transition set:

$$\widehat{\sigma}_{i \rightarrow j} = \sqrt{\frac{1}{M_{ij} - 1} \sum_{(b, \tau) \in \Omega_{ij}} \left(\text{MLP}_\psi(\mathbf{q}_{ij}^{(b, \tau)}) - \widehat{TE}_{i \rightarrow j} \right)^2}, \quad (11)$$

where larger $\widehat{\sigma}_{i \rightarrow j}$ indicates less stable directed dependency.

The scoring network MLP_ψ is trained jointly with the forecasting backbone in an end to end manner and is used only to produce relative dependency scores for graph construction, rather than to perform explicit density estimation.

3.3.2. Graph Construction with Sparsification

Based on the estimated directed dependency values, we construct a dynamic semantic graph $G = (V, E, W)$, where V is the semantic unit set, E denotes the directed edge set, and $W \in \mathbb{R}^{|V| \times |V|}$ is the weighted adjacency matrix. The edge weight from node i to node j is defined as:

$$W_{ij} = \max\left(0, \widehat{TE}_{i \rightarrow j} - \eta \widehat{\sigma}_{i \rightarrow j}\right), \quad (12)$$

where η is a significance coefficient used to suppress weak or unstable dependencies. Since the weighted adjacency matrix can become dense as the number of semantic units grows, we further sparsify the graph to improve robustness and efficiency. Sparsification is applied after computing all pairwise TE scores. Although this does not reduce the pairwise computation complexity, it reduces the cost of subsequent graph-based operations. Specifically, for each

node, we retain only the Top- K_g outgoing edges with the largest weights, where $K_g = \lceil \alpha \cdot N \rceil$ and N is the total number of semantic units:

$$W'_{ij} = \begin{cases} W_{ij}, & j \in \text{Top}K_g^i(W_{i \cdot}), \\ 0, & \text{otherwise,} \end{cases} \quad (13)$$

where $W_{i \cdot}$ denotes the i -th row of W . This step reduces graph density and suppresses noisy or marginal interactions. For subsequent aggregation, we normalize the sparse adjacency matrix by row:

$$\bar{W}_{ij} = \frac{W'_{ij}}{\sum_{j'} W'_{ij'} + \epsilon}, \quad (14)$$

where ϵ is a small positive constant for numerical stability.

3.3.3. Graph Clustering for Higher order Semantic Blocks

After obtaining the sparse semantic graph, we perform graph clustering to identify higher order semantic blocks. Let $c : V \rightarrow \{1, 2, \dots, K_b\}$ denote the cluster assignment function, where K_b is the number of semantic blocks. The optimal partition is obtained by maximizing directed weighted modularity, i.e., $C^* = \arg \max_C Q(C)$, with

$$Q(C) = \frac{1}{\mathcal{W}} \sum_{i, j} \left[W'_{ij} - \frac{s_i^{\text{out}} s_j^{\text{in}}}{\mathcal{W}} \right] \mathbf{1}(c_i = c_j), \quad (15)$$

where $s_i^{\text{out}} = \sum_j W'_{ij}$, $s_j^{\text{in}} = \sum_i W'_{ij}$, $\mathcal{W} = \sum_{i, j} W'_{ij}$, and $\mathbf{1}(\cdot)$ is the indicator function. In practice, clustering is performed online on the current sparse graph during each forward pass. The resulting assignments are treated as discrete structural decisions and are not backpropagated through. This decoupling does not affect global optimization since the clustering step contains no learnable parameters and receives no gradient signals, with all trainable parameters updated solely through the routing outputs, thereby avoiding non-smoothness in the loss landscape.

Each cluster defines a semantic block, denoted by $b_m = \{\mathbf{u}_i \mid c_i = m\}$ for $m = 1, 2, \dots, K_b$. To obtain a block level representation, we aggregate the units within each block using attention weights:

$$\omega_i = \frac{\exp(\phi_w(\mathbf{u}_i))}{\sum_{j \in b_m} \exp(\phi_w(\mathbf{u}_j))}, \quad i \in b_m, \quad (16)$$

where $\phi_w(\cdot)$ is a lightweight scoring function. The corresponding block embedding is defined as $\mathbf{e}_m = \sum_{i \in b_m} \omega_i \mathbf{u}_i$.

In addition, we define a block statistic vector $\mathbf{s}_m \in \mathbb{R}^{d_s}$ to summarize coarse structural properties of block m , including its normalized size, average incoming strength, average outgoing strength, and average temporal span. These low dimensional statistics provide auxiliary cues for routing beyond the content embedding $\mathbf{e}_m$.

To capture inter block context, we construct a block neighborhood graph induced by the sparse unit graph. Two

blocks m and n are considered neighbors if there exists at least one edge in W' linking a unit in b_m to a unit in b_n . The neighborhood of block m is denoted by $\mathcal{N}(m)$, and neighboring block embeddings are summarized by mean aggregation:

$$\text{Agg}(\{\mathbf{e}_n : n \in \mathcal{N}(m)\}) = \frac{1}{|\mathcal{N}(m)|} \sum_{n \in \mathcal{N}(m)} \mathbf{e}_n, \quad (17)$$

where the aggregation is defined as the zero vector if $\mathcal{N}(m) = \emptyset$. This design enables the higher order semantic structure to adapt to changing representations while maintaining stable and efficient optimization.

3.4. Importance Aware Routing

Semantic blocks in MTSF can exhibit diverse temporal characteristics. Slowly varying blocks tend to depend more on long range patterns, whereas rapidly varying blocks rely more on local structure. To accommodate this heterogeneity, we introduce an importance aware routing mechanism that routes semantic blocks to different experts.

3.4.1. Expert Network

Given the m -th semantic block, we first construct its routing representation by jointly incorporating block content, block level statistics, and neighborhood context:

$$\tilde{\mathbf{e}}_m = \mathbf{W}_e \mathbf{e}_m + \mathbf{W}_s \mathbf{s}_m + \mathbf{W}_n \text{Agg}(\{\mathbf{e}_n : n \in \mathcal{N}(m)\}) + \mathbf{b}_e, \quad (18)$$

where $\mathbf{W}_e$, $\mathbf{W}_s$, and $\mathbf{W}_n$ are projection matrices, and $\mathbf{b}_e$ is a bias term. We further introduce a semantic bias vector:

$$\mathbf{V}_m = \mathbf{V}_0 + \beta \cdot \tanh(\mathbf{W}_v \tilde{\mathbf{e}}_m + \mathbf{b}_v), \quad (19)$$

where $\mathbf{V}_0$ is a globally learnable base bias, β is a scaling coefficient, and $\mathbf{W}_v$ and $\mathbf{b}_v$ are learnable parameters.

We use R experts, denoted by $\{\text{Expert}_k\}_{k=1}^R$, each parameterized independently. For computation, each semantic block b_m is represented by its ordered unit sequence $\mathbf{U}_m = [\mathbf{u}_{m,1}, \mathbf{u}_{m,2}, \dots, \mathbf{u}_{m,|b_m|}]$, where the units are sorted by their temporal positions within the block. The k -th expert applies a block encoder $f_k(\cdot)$ to this ordered sequence and produces an expert specific block representation:

$$\mathbf{y}_m^{(k)} = \text{Expert}_k(b_m; \mathbf{V}_m) = f_k(\mathbf{U}_m) + \mathbf{U}_k \mathbf{V}_m, \quad (20)$$

where $f_k(\cdot)$ may be instantiated as a lightweight Transformer encoder or another sequence encoder, $\mathbf{U}_k$ is a learnable projection matrix, and $\mathbf{V}_m$ is the semantic bias vector associated with block m .

3.4.2. Routing Mechanism

The router computes expert assignment scores from the routing embedding of each semantic block. Specifically, the pre softmax routing score is $\tilde{\mathbf{s}}_m = \text{MLP}_r(\tilde{\mathbf{e}}_m)$, where $\tilde{\mathbf{s}}_m = [\tilde{s}_m^{(1)}, \dots, \tilde{s}_m^{(R)}]$ denotes the score vector over all experts. The normalized routing probability of expert k is then given by:

$$s_m^{(k)} = \frac{\exp(\tilde{s}_m^{(k)})}{\sum_{k'=1}^R \exp(\tilde{s}_m^{(k')})}. \quad (21)$$

Instead of using a fixed Top- K routing rule, we adopt Top- P routing so that the number of active experts adapts to the concentration of the routing distribution. When the distribution is concentrated, fewer experts are selected; when it is more diffuse, more experts are retained. Specifically, the experts are first sorted in descending order according to $\{s_m^{(k)}\}_{k=1}^R$, and we select the smallest expert set $\mathcal{T}_m$ such that $\sum_{k \in \mathcal{T}_m} s_m^{(k)} \geq P$, where $P \in (0, 1]$.

Based on the selected set, the sparsified routing weight is defined as:

$$\hat{s}_m^{(k)} = \begin{cases} \frac{s_m^{(k)}}{\sum_{k' \in \mathcal{T}_m} s_m^{(k')}}, & k \in \mathcal{T}_m, \\ 0, & \text{otherwise,} \end{cases} \quad (22)$$

where $\hat{s}_m^{(k)}$ denotes the normalized responsibility of expert k for semantic block m after Top- P selection. Finally, the outputs of the selected experts are aggregated to obtain the block level representation:

$$\mathbf{y}_m = \sum_{k=1}^R \hat{s}_m^{(k)} \mathbf{y}_m^{(k)}. \quad (23)$$

After processing all semantic blocks, the final forecasting result is generated by $\hat{\mathbf{Y}} = \text{Readout}(\{\mathbf{y}_m\}_{m=1}^{K_b})$, where $\text{Readout}(\cdot)$ denotes the regression head for final prediction.

3.5. Computational Complexity Analysis

The overall complexity of SCPaT is governed by three main components. For an input with C variables and sequence length L , let N_c denote the number of semantic units per variable, and let d_h be the hidden dimension. The Semantic Vector Encoder performs multi-scale convolutions with complexity $\mathcal{O}(C \cdot L \cdot d_h)$. The Transfer Entropy Graph Constructor computes pairwise dependency surrogates among semantic units, which scales as $\mathcal{O}(C^2 \cdot N_c^2 \cdot d_h)$. To manage the cost of subsequent operations, we apply static sparsification (Eq. (13)) to retain only the top- K_g edges per node, reducing the complexity of the routing and aggregation steps from $\mathcal{O}(C^2 N_c^2)$ to $\mathcal{O}(K_g C N_c)$. The routing module operates on K_b semantic blocks with R experts, contributing $\mathcal{O}(K_b \cdot R \cdot d_h^2)$. In our typical settings ($K_g = \alpha C N_c$ with $\alpha = 0.1$), the TE computation adds approximately 15%.

4. Experiment

4.1. Experimental Setup

4.1.1. Datasets

We conducted experiments on 12 widely used benchmark datasets. For long-term forecasting, we considered four ETT datasets, ETTh1, ETTh2, ETTm1, and ETTm2 (Zhou et al., 2021), which record electricity transformer temperature and load at hourly and minute-level resolutions. We also included Weather (Wu et al., 2021), which contains 21 meteorological variables, Traffic, which records road

Table 1

Summary of the 12 datasets used in our forecasting experiments, including the prediction horizons, data dimensionality, sampling frequency, and total number of time points.

Task Type	Dataset	Prediction Horizons	Time Point	Dimension	Frequency
Long-term Forecasting	ETTh1	{96,192,336,720}	17420	7	Hourly
	ETTh2	{96,192,336,720}	17420	7	Hourly
	ETTm1	{96,192,336,720}	69680	7	15 min
	ETTm2	{96,192,336,720}	69680	7	15 min
	Weather	{96,192,336,720}	52603	21	10 min
	Traffic	{96,192,336,720}	17451	862	Hourly
	Electricity	{96,192,336,720}	26211	321	Hourly
	Solar	{96,192,336,720}	52179	137	10 min
Short-term Forecasting	PEMS03	{12,24,48}	26208	358	5 min
	PEMS04	{12,24,48}	16992	307	5 min
	PEMS07	{12,24,48}	28224	883	5 min
	PEMS08	{12,24,48}	17856	170	5 min

occupancy rates on California freeways, Electricity, which tracks hourly electricity consumption for 321 households, and Solar, which contains solar power production records from photovoltaic plants. For short-term forecasting, we adopted four standard benchmarks from the PEMS collection, namely PEMS03, PEMS04, PEMS07, and PEMS08 (Zheng et al., 2020). These datasets consist of traffic flow measurements collected by road sensors. Table 1 provides detailed statistics for all datasets.

4.1.2. Baselines.

We compare the proposed method with nine representative baselines covering the main patch modeling strategies introduced above. These baselines include five Transformer-based models, PatchTST (Nie et al., 2023), iTransformer (Liu et al., 2024), DUET (Qiu et al., 2025), MSPatch (Cao et al., 2025), and Crossformer (Zhang and Yan, 2023b); one CNN-based model, TimesNet (Wu et al., 2022); one GNN-based model, MSGNet (Cai et al., 2024); one MLP-based model, HDMixer (Huang et al., 2024); and LSTM (Hochreiter and Schmidhuber, 1997). This selection allows us to evaluate the proposed method across different patch partitioning strategies and backbone architectures.

4.1.3. Implementation Details.

All baseline models and the proposed SCPaT are implemented in PyTorch 2.1.2 and evaluated under a unified experimental protocol on a single NVIDIA RTX 4090D GPU with 24 GB memory. For fair comparison, we re-trained all baselines on our dataset splits using their optimal hyperparameters as reported in the original papers, and performed grid search when unavailable. We follow the standard long-term forecasting setting and evaluate predictive performance at four representative forecasting horizons, $H \in \{96, 192, 336, 720\}$. For SCPaT, we perform a grid search (Pedregosa et al., 2011) over the semantic bias parameters to select the best configuration. Mean squared error (MSE) and mean absolute error (MAE) are reported as the evaluation metrics (Hyndman and Koehler, 2006).

4.2. Main Results

4.2.1. Long-term Forecasting.

Table 2 reports the long-term forecasting results of all compared methods. SCPaT achieves the best or second-best performance on the majority of datasets and forecasting horizons, demonstrating strong robustness across diverse long-term forecasting scenarios. Compared with representative methods based on fixed partitioning, multi-scale partitioning, and extendable partitioning, namely PatchTST, TimesNet, and HDMixer, SCPaT consistently yields lower prediction errors. These results support our central claim that semantic structured partitioning provides a more effective way to model complex temporal dynamics by preserving semantic consistency and capturing higher-order dependencies among heterogeneous temporal patterns. On the four ETT benchmarks, SCPaT reduces the average MSE by 4.6%, 7.1%, and 4.1% relative to PatchTST, TimesNet, and HDMixer, respectively. Moreover, compared with Crossformer, SCPaT also achieves consistently lower prediction errors across the long-term forecasting benchmarks.

4.2.2. Short-term Forecasting.

Table 3 presents the short-term forecasting results of all compared methods. SCPaT achieves the best or second-best performance on most datasets and forecasting horizons, indicating that the proposed framework generalizes well beyond the long-term forecasting setting. Although semantic structured partitioning is primarily introduced to address semantic inconsistency in long-horizon prediction, it also appears beneficial for short-term forecasting, where accurate modeling of local trend, periodic, and abrupt patterns remains important. These results suggest that organizing local segments into semantically meaningful units and emphasizing informative dependencies can also improve short-term prediction. Therefore, SCPaT consistently achieves lower prediction errors across the four PEMS benchmarks and attains the best overall performance.

Table 2

Multivariate long-term forecasting results over four prediction horizons, $H \in \{96, 192, 336, 720\}$, with the input length fixed at $L = 96$. The best and second-best results are marked in **bold red** and blue underline, respectively.

Models	Metric	SCPAT (Ours)		MSPatch (2025)		DUET (2025)		iTransformer (2024)		MSGNet (2024)		HDMixer (2024)		PatchTST (2023)		TimesNet (2023)		CrossFormer (2023)		LSTM (1997)	
		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	96	0.370	0.392	<u>0.372</u>	<u>0.395</u>	0.384	0.402	0.386	0.405	0.390	0.411	0.386	0.401	0.414	0.419	0.384	0.402	0.423	0.448	1.044	0.773
	192	0.418	0.422	<u>0.428</u>	<u>0.426</u>	0.437	0.429	0.441	0.436	0.442	0.442	0.441	0.430	0.460	0.445	0.446	0.439	0.471	0.474	1.217	0.832
	336	0.452	0.440	0.484	0.452	0.477	0.453	0.465	0.445	0.487	0.458	<u>0.458</u>	<u>0.443</u>	0.501	0.466	0.491	0.469	0.570	0.546	1.259	0.841
	720	0.449	0.458	<u>0.480</u>	<u>0.473</u>	0.486	0.475	0.503	0.491	0.494	0.488	0.512	0.484	0.500	0.488	0.521	0.500	0.653	0.621	1.271	0.838
ETTh2	96	0.288	0.338	<u>0.292</u>	<u>0.344</u>	0.298	0.351	0.297	0.349	0.328	0.371	0.293	0.338	0.302	0.348	0.340	0.374	0.745	0.584	2.522	1.278
	192	0.373	<u>0.396</u>	0.383	0.402	<u>0.374</u>	0.394	0.380	0.400	0.402	0.414	0.379	<u>0.396</u>	0.388	0.400	0.402	0.414	0.877	0.656	3.312	1.384
	336	0.408	0.427	<u>0.417</u>	<u>0.431</u>	<u>0.417</u>	<u>0.431</u>	0.428	0.432	0.435	0.443	0.426	0.444	0.426	0.433	0.452	0.452	1.043	0.731	3.291	1.388
	720	0.430	<u>0.444</u>	0.422	0.446	<u>0.427</u>	<u>0.442</u>	0.430	0.445	0.441	0.444	0.429	0.447	0.431	0.446	0.462	0.468	1.104	0.763	3.257	1.357
ETTm1	96	0.320	0.360	0.324	0.363	0.329	<u>0.361</u>	0.334	0.368	<u>0.323</u>	0.366	0.340	0.368	0.329	0.367	0.338	0.375	0.404	0.426	0.863	0.664
	192	0.362	0.383	0.367	<u>0.385</u>	<u>0.364</u>	<u>0.385</u>	0.377	0.391	0.376	0.397	0.382	0.386	0.367	<u>0.385</u>	0.374	0.387	0.450	0.451	1.113	0.776
	336	0.389	0.402	<u>0.390</u>	<u>0.404</u>	0.396	0.408	0.426	0.420	0.417	0.422	0.403	0.411	0.399	0.410	0.410	0.411	0.532	0.515	1.267	0.832
	720	<u>0.456</u>	0.436	0.454	<u>0.437</u>	0.469	<u>0.437</u>	0.491	0.459	0.481	0.458	0.473	0.439	0.454	0.439	0.478	0.450	0.666	0.589	1.324	0.858
ETTm2	96	0.173	0.258	<u>0.175</u>	0.260	0.179	0.262	0.180	0.264	0.181	0.262	0.183	0.266	0.181	<u>0.259</u>	0.187	0.267	0.287	0.366	2.041	1.073
	192	0.238	0.300	<u>0.239</u>	<u>0.302</u>	0.240	0.300	0.250	0.309	0.247	0.307	0.247	0.307	0.241	<u>0.302</u>	0.249	0.309	0.414	0.492	2.249	1.112
	336	0.297	0.339	0.297	<u>0.340</u>	<u>0.302</u>	0.341	0.311	0.348	0.312	0.346	0.305	0.345	0.305	0.343	0.321	0.351	0.597	0.543	2.568	1.238
	720	0.394	0.394	<u>0.396</u>	<u>0.397</u>	0.399	<u>0.397</u>	0.412	0.407	0.414	0.403	0.406	0.400	0.402	0.406	0.408	0.403	1.730	1.042	2.720	1.287
Weather	96	0.163	0.208	<u>0.165</u>	<u>0.209</u>	0.167	<u>0.209</u>	0.174	0.214	0.169	0.212	0.174	0.220	0.177	0.218	0.172	0.220	0.171	0.230	0.369	0.406
	192	0.209	0.250	<u>0.211</u>	<u>0.252</u>	0.212	0.254	0.221	0.254	0.218	0.255	0.225	0.263	0.225	0.259	0.219	0.261	0.226	0.277	0.416	0.435
	336	0.266	0.292	<u>0.268</u>	<u>0.294</u>	0.269	0.297	0.278	0.296	0.272	0.299	0.277	0.301	0.278	0.297	0.280	0.306	0.272	0.335	0.455	0.454
	720	<u>0.345</u>	0.349	0.342	0.345	0.348	<u>0.347</u>	0.358	0.349	0.350	0.348	0.349	<u>0.347</u>	0.354	0.348	0.365	0.359	0.398	0.418	0.535	0.520
Traffic	96	0.393	0.247	0.460	0.295	<u>0.395</u>	<u>0.256</u>	<u>0.395</u>	0.268	0.605	0.344	0.529	0.353	0.544	0.359	0.593	0.321	0.522	0.290	0.843	0.453
	192	0.425	0.259	0.466	0.300	<u>0.420</u>	<u>0.266</u>	0.417	0.276	0.613	0.359	0.535	0.361	0.540	0.354	0.617	0.336	0.530	0.293	0.847	0.453
	336	0.454	0.270	0.484	0.306	<u>0.458</u>	<u>0.272</u>	0.461	0.283	0.642	0.376	0.541	0.361	0.551	0.358	0.629	0.336	0.558	0.305	0.853	0.455
	720	0.489	<u>0.291</u>	0.510	0.323	0.499	0.288	<u>0.497</u>	0.302	0.702	0.401	0.591	0.389	0.586	0.375	0.640	0.350	0.589	0.328	1.500	0.805
Electricity	96	0.139	0.233	0.158	0.257	<u>0.148</u>	<u>0.236</u>	0.149	0.240	0.165	0.274	0.163	0.275	0.195	0.285	0.168	0.272	0.219	0.314	0.375	0.437
	192	0.155	0.248	0.171	0.268	0.164	<u>0.249</u>	<u>0.162</u>	0.253	0.184	0.292	0.186	0.281	0.195	0.285	0.184	0.289	0.231	0.322	0.442	0.473
	336	0.168	0.263	0.184	0.283	<u>0.178</u>	<u>0.265</u>	<u>0.178</u>	0.269	0.195	0.302	0.199	0.295	0.215	0.305	0.198	0.300	0.246	0.337	0.439	0.473
	720	0.200	0.293	0.223	0.308	<u>0.206</u>	<u>0.303</u>	0.225	0.317	0.231	0.332	0.231	0.321	0.256	0.337	0.220	0.320	0.280	0.363	0.980	0.814
Solar	96	0.202	<u>0.234</u>	<u>0.204</u>	0.237	0.205	0.217	0.203	0.237	0.208	0.243	0.207	0.244	0.234	0.286	0.250	0.292	0.310	0.331	0.663	0.697
	192	0.228	<u>0.258</u>	<u>0.231</u>	0.260	<u>0.231</u>	0.239	0.233	0.261	0.258	0.281	0.245	0.273	0.267	0.310	0.296	0.318	0.734	0.725	0.686	0.743
	336	0.239	<u>0.270</u>	<u>0.240</u>	0.273	0.243	0.242	0.248	0.273	0.293	0.311	0.263	0.285	0.290	0.315	0.319	0.330	0.750	0.735	0.713	0.785
	720	0.250	0.278	0.250	<u>0.275</u>	0.255	0.256	0.260	0.280	0.290	0.315	0.288	0.299	0.289	0.317	0.338	0.337	0.769	0.765	0.775	0.812
1 st Count		28	24	<u>5</u>	1	0	<u>7</u>	1	0	0	0	0	1	1	0	0	0	0	0	0	0

4.3. Model Analysis

4.3.1. Routing Analysis.

We analyze the routing behavior of the Importance Aware Router by averaging the expert assignment probabilities over all semantic blocks extracted from the test set, as reported in Table 4. Expert 1 captures trend-dominated blocks, Expert 2 specializes in periodic patterns, and Expert 3 handles high-frequency fluctuations. The results reveal a clear dataset-dependent allocation pattern. For ETTh1 and ETTm1, the routing weights are more evenly distributed across the three experts (0.62/0.21/0.17 and 0.51/0.25/0.24, respectively), suggesting that these datasets require complementary modeling of multiple dependency patterns. In contrast, for Weather, the routing probability is concentrated on Expert 1 (0.81), while Expert 2 and Expert 3 receive much smaller weights. This difference indicates that the router adapts its allocation strategy to dataset characteristics, providing empirical support for the effectiveness of the proposed routing mechanism.

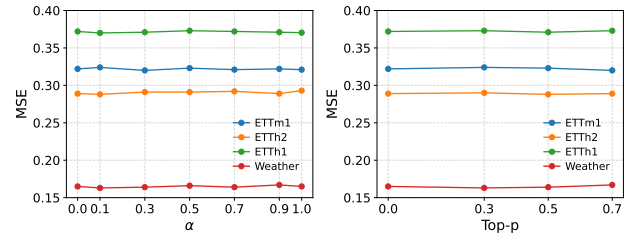


Figure 3: Sensitivity analysis of the hyperparameters α and Top-P on ETTm1, ETTh1, ETTh2, and Weather, with both the input length and prediction horizon fixed at 96.

4.3.2. Hyperparameter Sensitivity

We further analyze the sensitivity of SCPAT to the static sparsification ratio α and the routing threshold Top-P, as shown in Figure 3. SCPAT remains stable across a broad range of hyperparameter values. For α , the optimal value varies across datasets: ETTh1 and ETTh2 perform best at $\alpha = 0.1$, ETTm1 at $\alpha = 0.3$, and Weather at $\alpha = 0.1$. This suggests that datasets with stronger local fluctuations or

Table 3

Multivariate short-term forecasting results over three prediction horizons, $H \in \{12, 24, 48\}$, with the input length fixed at $L = 96$. The best and second-best results are marked in **bold red** and blue underline, respectively.

Models	Metric		SCPAT (Ours)		MSPatch (2025)		DUET (2025)		iTransformer (2024)		MSGNet (2024)		HDMixer (2024)		TimesNet (2023)		PatchTST (2023)		CrossFormer (2023)	
			MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
PEMS03	12		0.066	0.169	0.074	0.179	<u>0.071</u>	0.176	<u>0.071</u>	<u>0.174</u>	0.083	0.197	0.076	0.184	0.085	0.192	0.099	0.216	0.090	0.203
	24		<u>0.086</u>	0.195	0.097	0.200	0.089	<u>0.197</u>	0.093	0.201	0.097	0.217	0.095	0.210	0.118	0.223	0.142	0.259	0.121	0.240
	48		<u>0.122</u>	<u>0.235</u>	0.131	0.244	0.118	0.229	0.125	0.236	0.135	0.241	0.122	0.245	0.155	0.260	0.211	0.319	0.202	0.317
PEMS04	12		0.073	0.176	<u>0.075</u>	0.185	0.076	<u>0.179</u>	0.078	0.183	0.082	0.187	0.084	0.188	0.087	0.195	0.105	0.224	0.098	0.218
	24		0.091	0.199	<u>0.093</u>	<u>0.201</u>	0.097	0.203	0.095	0.205	0.128	0.241	0.097	0.210	0.103	0.215	0.153	0.257	0.131	0.256
	48		<u>0.124</u>	0.237	0.134	0.244	0.119	<u>0.239</u>	0.126	0.240	0.192	0.315	0.134	0.253	0.136	0.250	0.229	0.339	0.205	0.326
PEMS07	12		0.058	0.153	0.066	<u>0.162</u>	<u>0.059</u>	0.166	0.067	0.165	0.087	0.191	0.074	0.182	0.082	0.181	0.095	0.207	0.094	0.200
	24		0.074	0.174	0.083	0.189	<u>0.079</u>	<u>0.183</u>	0.088	0.190	0.135	0.241	0.089	0.197	0.101	0.204	0.150	0.262	0.139	0.247
	48		0.102	0.207	0.112	0.217	<u>0.106</u>	<u>0.211</u>	0.110	0.215	0.297	0.351	0.124	0.217	0.134	0.238	0.253	0.340	0.311	0.369
PEMS08	12		0.070	0.173	0.079	<u>0.175</u>	<u>0.077</u>	0.177	0.079	0.182	0.159	0.208	0.098	0.193	0.112	0.212	0.168	0.232	0.165	0.214
	24		0.085	0.187	<u>0.094</u>	0.211	0.098	<u>0.201</u>	0.115	0.219	0.210	0.251	0.108	0.223	0.141	0.238	0.224	0.281	0.215	0.260
	48		0.112	0.220	0.150	0.232	<u>0.139</u>	<u>0.227</u>	0.186	0.235	0.311	0.358	0.167	0.247	0.198	0.283	0.321	0.354	0.315	0.335
1 st Count			9	11	0	0	<u>3</u>	<u>1</u>	0	0	0	0	0	0	0	0	0	0	0	0

Table 4

Average routing probability distribution across experts, with both the input length and prediction horizon fixed at 96.

Dataset	Expert 1	Expert 2	Expert 3
ETTh1	0.62	0.21	0.17
ETTh1	0.51	0.25	0.24
Weather	0.81	0.11	0.08

denser interactions benefit from larger α , whereas datasets with more regular structures prefer relatively sparser graphs. A similar trend is observed for Top-P. ETTh1 performs best at Top-P = 0.5, Weather at Top-P = 0.3, and ETTh1 around Top-P = 0.7. When Top-P is too small, routing becomes overly selective; when it is too large, expert activation becomes overly uniform, weakening expert specialization. These results suggest that SCPaT performs best under moderate sparsity in both graph construction and expert routing.

4.3.3. Robustness to Irregular and Noisy Data.

We evaluate the robustness of the model under incomplete observation scenarios by introducing missing values through zero padding. Specifically, according to different missing rates ranging from 0% to 30%, some input data points are randomly masked, the masked entries are filled with 0, and the remaining observations are kept unchanged. The results are shown in Table 5. Under different missing rates, SCPaT consistently achieves lower forecasting errors compared with baseline methods. As the missing rate increases, the performance of all models gradually degrades, but SCPaT, relying on its semantic structured modeling mechanism, can capture the dependencies among the remaining observations, alleviate the impact of missing information, and thus maintain relatively stable performance.

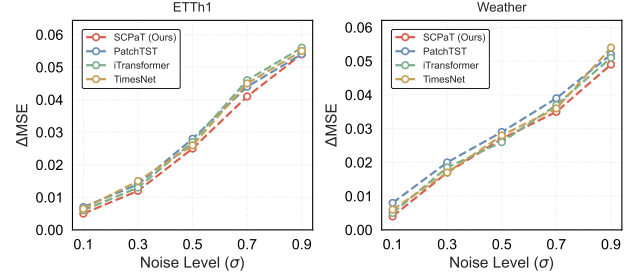


Figure 4: Performance degradation under different Gaussian noise levels on ETTh1 and Weather with both the input length and prediction horizon fixed at 96.

To further evaluate robustness against input perturbations, we add zero-mean Gaussian noise to the test sequences with standard deviation $\sigma \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$. Figure 4 compares the performance degradation of SCPaT, PatchTST, iTransformer, and TimesNet under increasing noise levels. As σ increases, the MSE of all models rises. SCPaT consistently exhibits the slowest performance decay across all noise levels, confirming the robustness of semantic partitioning against high frequency input perturbations.

4.3.4. Look-back Window.

Generally, increasing the input sequence length can provide richer historical context for forecasting. However, most existing models do not consistently benefit from longer input sequences, and their performance often fluctuates across different look-back windows (Shao et al., 2024). As shown in Figure 5, longer input sequences do not always lead to better results and may introduce more redundant or noisy information. In contrast, SCPaT maintains consistently strong and stable performance across different look-back window settings, suggesting that it can better balance local dynamics and long-term dependencies.

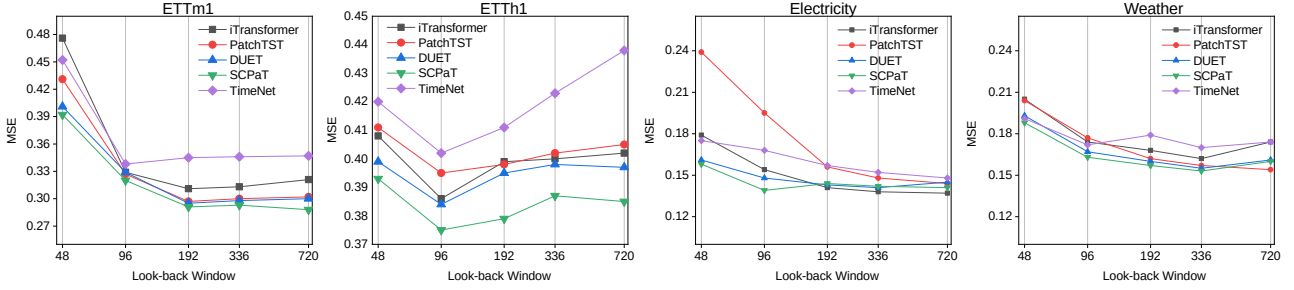


Figure 5: Performance comparison under different look-back window lengths on ETTm1, ETTh1, Electricity, and Weather, with $L \in \{48, 96, 192, 336, 720\}$ and the prediction horizon fixed at 96.

Table 5

Robustness analysis on the ETTm2 and ETTh1 datasets under different missing rates, with the input length fixed at $L = 96$ and the prediction horizon fixed at $H = 96$. The best results are highlighted in bold.

Missing Rate	ETTM2						ETTh1					
	SCPaT (Ours)		iTransformer		PatchTST		SCPaT (Ours)		iTransformer		PatchTST	
	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
0.00	0.173	0.258	0.180	0.264	0.181	0.259	0.370	0.392	0.386	0.405	0.414	0.419
0.05	0.213	0.297	0.249	0.441	0.252	0.327	0.379	0.401	0.404	0.413	0.436	0.428
0.10	0.248	0.326	0.295	0.472	0.328	0.376	0.390	0.412	0.433	0.424	0.455	0.441
0.15	0.287	0.354	0.379	0.559	0.411	0.423	0.404	0.424	0.462	0.432	0.467	0.460
0.20	0.332	0.385	0.433	0.567	0.502	0.471	0.419	0.438	0.478	0.448	0.482	0.483
0.25	0.388	0.419	0.506	0.574	0.601	0.520	0.437	0.454	0.487	0.473	0.498	0.503
0.30	0.452	0.456	0.596	0.580	0.709	0.569	0.459	0.471	0.500	0.498	0.511	0.527

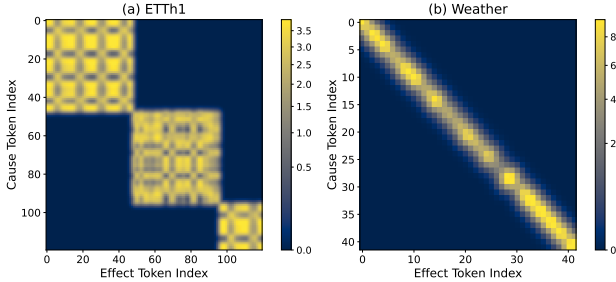


Figure 6: Heatmaps of the learned directed adjacency matrix A_{final} on the ETTh1 and Weather datasets.

4.4. Case Study and Visualization

4.4.1. Adjacency Matrix Interpretation.

To further understand how SCPaT captures structural dependencies, we visualize the final adjacency matrix A_{final} learned by the semantic graph constructor, with the input length fixed at $L = 96$ and the prediction horizon fixed at $H = 96$, as shown in Figure 6. The matrix encodes the directed dependency strength between semantic patches after static sparsification and importance aware routing, where each entry represents the information propagation strength from a source patch to a target patch. On ETTh1, the adjacency matrix exhibits a clear block-wise structure, with strong interactions both within variable-specific segments and across certain groups of patches. This pattern

suggests that SCPaT can jointly capture intra-variable temporal continuity and structured inter-variable coupling. In contrast, on Weather, the adjacency matrix is dominated by a near-diagonal pattern, with only limited activation away from the diagonal. This suggests that most tokens depend primarily on temporally aligned counterparts rather than complex cross-token interactions, which is consistent with the more regular and periodic nature of meteorological data.

4.4.2. Visualization of Forecasting Results.

Figure 7 provides a qualitative comparison between SCPaT and several representative baselines. SCPaT produces predictions that are more closely aligned with the ground truth across the three examples. It more faithfully captures the periodic structure and amplitude variation of the target sequence, while also tracking local trend changes more accurately. In oscillatory regions, SCPaT better preserves the phase and waveform shape of the ground truth, whereas the baseline methods show more noticeable deviations, particularly near turning points and local extrema. These results indicate that SCPaT can simultaneously capture global temporal regularities and local dynamics, which is consistent with its stronger quantitative forecasting performance.

4.5. Ablation Study

To assess the effectiveness of the core components in SCPaT, we conduct ablation studies on three representative datasets, namely ETTm2, Weather, and Traffic,

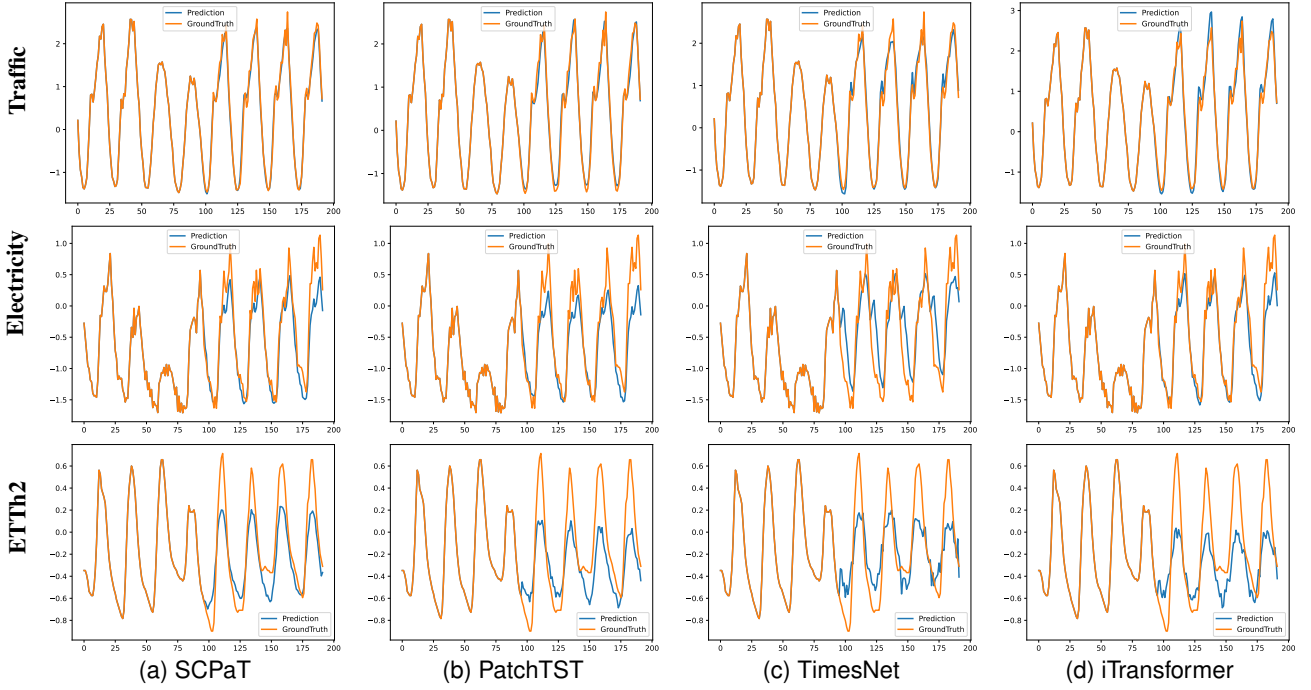


Figure 7: Visualization of prediction results on Electricity, Traffic, and ETTh2 under the Lookback-96-Horizon-96 setting.

Table 6

Ablation results of the SCPaT components on ETTm2, Weather, and Traffic over prediction horizons $H \in \{96, 192, 336, 720\}$, with the input length fixed at $L = 96$. The best and second-best results are marked in **bold red** and **blue underline**, respectively.

Dataset		ETTh2				Weather				Traffic			
Prediction Length		96	192	336	720	96	192	336	720	96	192	336	720
SCPaT	MSE	0.173	0.238	0.297	0.394	0.163	0.209	0.266	0.345	0.393	0.425	0.454	0.489
	MAE	0.258	0.300	0.339	0.394	0.208	0.250	0.292	0.349	0.247	0.259	0.270	0.291
w/o SV Encoder	MSE	0.196	0.259	0.317	0.412	0.204	0.248	0.302	0.373	0.414	0.448	0.478	0.510
	MAE	0.278	0.318	0.354	0.408	0.254	0.287	0.323	0.367	0.274	0.281	0.294	0.316
w/o IAR	MSE	<u>0.178</u>	<u>0.245</u>	<u>0.302</u>	<u>0.400</u>	<u>0.167</u>	0.216	<u>0.273</u>	<u>0.353</u>	<u>0.397</u>	<u>0.431</u>	<u>0.459</u>	<u>0.495</u>
	MAE	<u>0.264</u>	<u>0.309</u>	0.347	<u>0.402</u>	<u>0.214</u>	<u>0.257</u>	<u>0.299</u>	<u>0.354</u>	0.253	0.267	<u>0.274</u>	<u>0.297</u>
w/o TE Graph	MSE	0.180	0.247	0.309	0.407	0.171	<u>0.214</u>	0.277	0.356	0.401	0.436	0.466	0.498
	MAE	0.270	0.311	<u>0.343</u>	0.413	0.221	0.262	0.301	0.355	<u>0.249</u>	<u>0.264</u>	0.277	0.301

which represent settings with low, medium, and high dimensionality, respectively. The considered variants are **W/O SV Encoder**, which removes the semantic vector encoder module; **W/O IAR**, which removes the importance aware routing mechanism; and **W/O TE Graph**, which removes the transfer entropy graph construction module. Table 6 reports the ablation results. The full SCPaT consistently achieves the best performance across all datasets and prediction horizons, while removing any individual component degrades forecasting performance. In particular, removing semantic vector encoder or importance aware routing leads to larger performance drops in most settings, indicating their importance for capturing semantic variation within patches and emphasizing informative temporal patterns. The

degradation without the TE Graph further highlights the role of modeling directed dependencies among semantic units.

4.6. Model Efficiency

We compare SCPaT with representative baselines in terms of forecasting performance, training time, and memory consumption under the official model configurations. Figure 8 illustrates the trade off between efficiency and performance on the ETTm2 and Weather datasets, with the input length fixed at $L = 96$ and the prediction horizon fixed at $H = 96$. The horizontal axis denotes training time, the vertical axis denotes MSE, and the bubble size indicates peak GPU memory usage during training. SCPaT achieves competitive MSE with moderate training time and

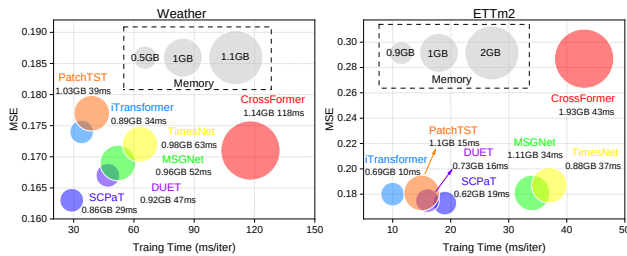


Figure 8: Model efficiency comparison of different methods on ETTm2 and Weather.

memory consumption, suggesting that it remains efficient while preserving strong predictive performance.

5. Conclusion

This paper revisits patch based MTSF from the perspective of semantic structure. Rather than relying solely on fixed, multi-scale, or extendable partitioning schemes, we argue that effective forecasting also requires explicitly modeling the semantic consistency of local segments and the higher order dependencies among them. Based on this view, we propose SCPaT, which integrates semantic structured partitioning, transfer entropy based dependency modeling, and importance aware routing into a unified framework. Extensive experiments show that SCPaT achieves strong and consistent improvements across diverse forecasting settings. More importantly, the results suggest that incorporating semantic structure into patch based modeling provides a promising direction for handling heterogeneous temporal patterns and complex multivariate dependencies. We believe future work can further explore more efficient graph construction strategies and scalable structured modeling to broaden its applicability.

Generative AI disclosure

The authors used artificial intelligence tools (e.g., ChatGPT) solely for language polishing and grammar correction. All research concepts, methodology, and visual elements were developed independently without AI assistance. The authors take full responsibility for the scholarly content and academic integrity of the work.

Acknowledgments

This work was supported in part by the Key Research Project Plan for Basic Research Special Fund of Higher Education Institutions in Henan Province under Grant No.25ZX009, and by the Henan Provincial Natural Science Foundation under Grant No.262300422534.

References

Cai, W., Liang, Y., Liu, X., Feng, J., Wu, Y., 2024. Msgnet: Learning multi-scale inter-series correlations for multivariate time series forecasting, in: Proceedings of the AAAI conference on artificial intelligence.

Cao, Y., Tian, Z., Guo, W., Liu, X., 2025. Mspatch: A multi-scale patch mixing framework for multivariate time series forecasting. Expert Systems with Applications .

Chen, P., Zhang, Y., Cheng, Y., Shu, Y., Wang, Y., Wen, Q., Yang, B., Guo, C., 2024. Pathformer: Multi-scale transformers with adaptive pathways for time series forecasting. arXiv preprint arXiv:2402.05956 .

Gasparin, A., Lukovic, S., Alippi, C., 2022. Deep learning for time series forecasting: The electric load case. CAAI Transactions on Intelligence Technology .

Hochreiter, S., Schmidhuber, J., 1997. Long short-term memory. Neural Computation .

Hu, Y., Liu, P., Zhu, P., Cheng, D., Dai, T., 2025. Adaptive multi-scale decomposition framework for time series forecasting, in: Proceedings of the AAAI Conference on Artificial Intelligence.

Huang, Q., Shen, L., Zhang, R., Cheng, J., Ding, S., Zhou, Z., Wang, Y., 2024. Hdmixer: Hierarchical dependency with extendable patch for multivariate time series forecasting, in: Proceedings of the AAAI conference on artificial intelligence.

Huang, Z., Zheng, R., Zhu, J., Liu, L., Li, M., Liu, M., 2025. Endexformer: Hierarchical endogenous-exogenous synergy for multivariate time series forecasting, in: ECAI.

Hyndman, R.J., Koehler, A.B., 2006. Another look at measures of forecast accuracy. International journal of forecasting .

Karevan, Z., Suykens, J.A., 2020. Transductive lstm for time-series prediction: An application to weather forecasting. Neural Networks .

Kazemi, M.H., Shiri, J., Marti, P., Majnooni-Heris, A., 2020. Assessing temporal data partitioning scenarios for estimating reference evapotranspiration with machine learning techniques in arid regions. Journal of Hydrology .

Lippi, M., Bertini, M., Frasconi, P., 2013. Short-term traffic flow forecasting: An experimental comparison of time-series analysis and supervised learning. IEEE Transactions on Intelligent Transportation Systems .

Liu, H., Yang, C., Zhu, X., et al., 2025a. Semantic-enhanced time-series forecasting via large language models. arXiv preprint arXiv:2508.07697 .

Liu, Y., Hu, T., Zhang, H., Wu, H., Wang, S., Ma, L., Long, M., 2024. itransformer: Inverted transformers are effective for time series forecasting, in: International Conference on Learning Representations.

Liu, Y., Wu, H., Wang, J., Long, M., 2022. Non-stationary transformers: Exploring the stationarity in time series forecasting. Advances in neural information processing systems .

Liu, Z., Duan, P., Wang, B., Tang, X., Chu, Q., Zhang, C., Huang, Y., Zhang, B., 2025b. Disms-ts: Eliminating redundant multi-scale features for time series classification, in: Proceedings of the ACM International Conference on Multimedia.

Nie, Y., Nguyen, N.H., Sinthong, P., Kalagnanam, J., 2023. A time series is worth 64 words: Long-term forecasting with transformers, in: International Conference on Learning Representations.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., et al., 2011. Scikit-learn: Machine learning in python. the Journal of machine Learning research .

Piao, X., Chen, Z., Murayama, T., Matsubara, Y., Sakurai, Y., 2024. Fredformer: Frequency debiased transformer for time series forecasting, in: Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining.

Qiu, X., Hu, J., Zhou, L., Wu, X., Du, J., Zhang, B., Guo, C., Zhou, A., Jensen, C.S., Sheng, Z., Yang, B., 2024. TFB: Towards comprehensive and fair benchmarking of time series forecasting methods, in: Proc. VLDB Endow., pp. 2363–2377.

Qiu, X., Wu, X., Lin, Y., Guo, C., Hu, J., Yang, B., 2025. Duet: Dual clustering enhanced multivariate time series forecasting, in: ACM SIGKDD Conference on Knowledge Discovery and Data Mining.

Qiu, X., Zhu, Y., Li, Z., Wu, X., Yang, B., Hu, J., 2026. Dag: A dual correlation network for time series forecasting with exogenous variables, in: ICML.

Shao, Z., Wang, F., Xu, Y., Wei, W., Yu, C., Zhang, Z., Yao, D., Sun, T., Jin, G., Cao, X., et al., 2024. Exploring progress in multivariate time series

- forecasting: Comprehensive benchmarking and heterogeneity analysis. *IEEE Transactions on Knowledge and Data Engineering* .
- Wu, H., Hu, T., Liu, Y., Zhou, H., Wang, J., Long, M., 2022. Timesnet: Temporal 2d-variation modeling for general time series analysis. *arXiv preprint arXiv:2210.02186* .
- Wu, H., Xu, J., Wang, J., Long, M., 2021. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Advances in neural information processing systems* .
- Wu, X., Qiu, X., Li, Z., Wang, Y., Hu, J., Guo, C., Xiong, H., Yang, B., 2025. CATCH: Channel-aware multivariate time series anomaly detection via frequency patching, in: *ICLR*.
- Yang, Y., Ding, W., Gu, Y., Zhang, H., Wei, W., Gao, H., 2025. Beyond data heterogeneity: A multivariate time series forecasting for energy systems through enhanced channel fusion in frequency domain. *Information Fusion* .
- Ye, C., Ma, Q., 2023. Multi-granularity framework for unsupervised representation learning of time series. *arXiv preprint arXiv:2312.07248* .
- Zhang, X., Wang, J., Bai, Y., Zhang, L., Lin, Y., 2025. Tf4tf: Multi-semantic modeling within the time–frequency domain for long-term time-series forecasting. *Neurocomputing* .
- Zhang, Y., Yan, J., 2023a. Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting, in: *International conference on learning representations*.
- Zhang, Y., Yan, J., 2023b. Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting, in: *International Conference on Learning Representations*.
- Zheng, C., Fan, X., Wang, C., Qi, J., 2020. Gman: A graph multi-attention network for traffic prediction, in: *Proceedings of the AAAI conference on artificial intelligence*.
- Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., Zhang, W., 2021. Informer: Beyond efficient transformer for long sequence time-series forecasting, in: *Proceedings of the AAAI conference on artificial intelligence*.
- Zhou, T., Ma, Z., Wen, Q., Wang, X., Sun, L., Jin, R., 2022. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting, in: *International conference on machine learning*.